

Part I

In this part we compare the Jacobi method and the Gauss-Seidel method.

First modify the M-file GS.m from Lab 10 to create the M-file Jacobi.m that can solve a system $\mathbf{Ax} = \mathbf{b}$ iteratively by the Jacobi method. The first line of your M-file should be

```
function [x,rel,niter]=Jacobi(A,b,x0,tol,n)
```

(Hint: You only need to change one line of GS.m in order to generate Jacobi.m.)

For many systems the Gauss-Seidel method performs better than the Jacobi method. This is because the lower triangular part of $\mathbf{A}$ is a better approximation of $\mathbf{A}$ than the diagonal part of $\mathbf{A}$. Another point of view is that the Gauss-Seidel method is the Jacobi method with immediate updates.

Consider the row strictly diagonal dominant system

$$(*) \quad \begin{bmatrix} 5 & 0 & 1 \\ -1 & 5 & 1 \\ 0 & 1 & 5 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} = \begin{bmatrix} 8 \\ -3 \\ 14 \end{bmatrix}.$$

The exact solution of (*) is $[1 \ -1 \ 3]^t$.

Create the coefficient matrix $\mathbf{A}$ and the right-hand side vector $\mathbf{b}$ of (*), and type the following commands:

```
>>format long e
```

```
>>x0=zeros(3,1);
```

This creates the zero vector as the initial guess.

```
>>tol=1.e-6;
```

The tolerance for the estimated relative error is set to be 10^{-6} .

First we solve (*) by Jacobi.m.

```
>>[x,rel,niter]=Jacobi(A,b,x0,tol,20)
```

Next we solve (*) by GS.m.

```
>>[x,rel,niter]=GS(A,b,x0,tol,20)
```

Clearly for the system (*) the Gauss-Seidel method performs better than the Jacobi method.

However, in general, it can happen that the Jacobi method converges but the Gauss-Seidel method does not. Consider the system

$$(\dagger) \quad \begin{bmatrix} 1 & 2 & -2 \\ 1 & 1 & 1 \\ 2 & 2 & 1 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} = \begin{bmatrix} -3 \\ 1 \\ 1 \end{bmatrix}.$$

The exact solution of $(\dagger)$ is $[1 \ -1 \ 1]^t$.

Create the coefficient matrix **A** and the right-hand side vector **b** of $(\dagger)$, and reset the initial guess as follows.

```
>>x0=rand(3,1);
```

This creates a random initial guess and we will use the same tolerance 10^{-6} from above.

```
>>[x,rel,niter]=Jacobi(A,b,x0,tol,20)
```

The Jacobi method solves $(\dagger)$ exactly in 4 steps! (Actually the exact solution is reached after three iterations but the estimated relative error after three iterations still exceeds the tolerance.)

```
>>[x,rel,niter]=GS(A,b,x0,tol,20)
```

Clearly the Gauss-Seidel method fails to converge for $(\dagger)$. Note also that the estimate for the relative error is meaningless because the method does not converge.

It can also happen that the Gauss-Seidel method converges but the Jacobi method does not. Consider the system

$$(\diamond) \quad \begin{bmatrix} 2 & 1 & 2 \\ 1 & 1 & 0 \\ 1 & 1 & 1 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} = \begin{bmatrix} 10 \\ 3 \\ 6 \end{bmatrix}.$$

The exact solution of $(\diamond)$ is $[1 \ 2 \ 3]^t$.

Create the coefficient matrix **A** and the right-hand side vector **b** of $(\diamond)$. We will use the same random initial guess and tolerance from above.

```
>>[x,rel,niter]=GS(A,b,x0,tol,30)
```

We see that the Gauss-Seidel method converges for $(\diamond)$.

```
>>[x,rel,niter]=Jacobi(A,b,x0,tol,30)
```

Clearly the Jacobi method does not converge for $(\diamond)$.

Clear all variables before you work on Part II.